

Image Classification Using Convolutional Neural Networks (CNN)

Group

Michael A. Libio

Alejanro Zahinos

Baggiyam Shanmugam

Project Overview

Project Description

- The goal of this project is to build a **Convolutional Neural Network (CNN)** to classify images from the **CIFAR-10 dataset** into 10 object categories.

Dataset can be downloaded directly via `tensorflow.keras.datasets.cifar10`.

CIFAR-10 Dataset Overview

The dataset used for this project is CIFAR-10, a popular benchmark in computer vision tasks.

- Total Images: 60,000 color images
- Image Size: 32 x 32 pixels, RGB
- Training Set: 50,000 images
- Test Set: 10,000 images
- Classes (10): airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck
- Images per Class: 6,000



airplane



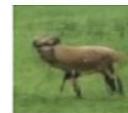
automobile



bird



cat



deer



dog



frog



truck

CNN Model Development

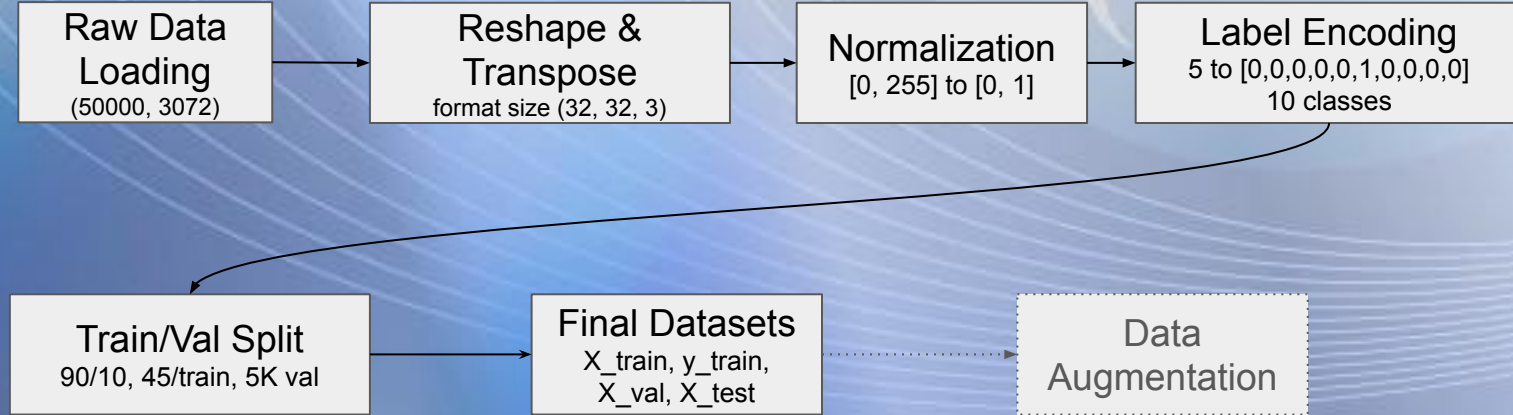
CNN-based architecture(Base_Model) : A simple convolutional neural network with two Conv2D layers, max-pooling, and dense layers for classification. Ideal for small datasets or educational purposes.



What Was Done

- Preprocessed the images (normalization + one-hot encoding).
- Built a CNN model from scratch.
- Trained and validated the model over multiple epochs.
- Evaluated performance on unseen test data.

Data Preprocessing



CNN-based architecture

It's an Evolution)

Model	Key Features	Parameters	Accuracy	Training Time
Baseline	2 conv blocks, no augmentation	545K	70.74%	65s
ResNet-inspired	Skip connections, BatchNorm	319K	82.65%	724s
VGG-style	Paired convolutions, GlobalAvgPool	307K	84.46%	1187s
Deep CNN	Larger filters, stronger augmentation	3,253K	89.76%	1184s

Key Techniques & Performance Impact

Most Impactful Techniques:

- BatchNormalization (+12% accuracy)
- Data augmentation (even mild: +12-14%)
- Paired convolutions/skip connections
- Progressive dropout (0.25→0.25→0.5)

Class Performance Gains (F1-Score, Baseline → Deep):

- Most improved: Bird (+0.27), Cat (+0.25), Deer (+0.25), Dog (+0.25)
- Least improved: Ship (+0.12), Truck/Auto/Horse (+0.13)

Conclusion: Modern CNN techniques yield dramatic improvements with challenging classes benefiting most

Optimization Journey: From Baseline to 89% Accuracy

Sequential Optimization Steps

1. Architecture Refinement

- Added BatchNormalization after every conv layer
- Implemented paired convolutions
- Introduced skip connections
- Transitioned to GlobalAveragePooling

2. Regularization Strategy

- Moved from single dropout (0.5) to progressive dropout (0.25 → 0.25 → 0.5)
- Added BatchNorm to dense layers
- Implemented early stopping

3. Data Augmentation Progression

- Introduced mild augmentation
- Scaled to moderate augmentation
- Result: Doubled effective training data while improving generalization

4. Parameter Efficiency Optimization

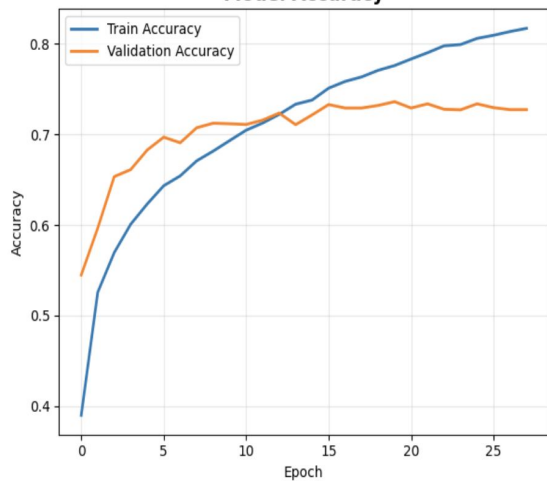
- Reduced parameters by 45% (545K → 307K) while increasing accuracy by 14%
- Strategically increased capacity in final model where most beneficial
- Found optimal balance: VGG-style architecture with 307K parameters (84.46%)
- Pushed performance ceiling: Deep CNN with 3.25M parameters (89.10%)

Key Insight: Most improvement came from modern CNN techniques (BatchNorm, augmentation), not from adding parameters. Strategic optimization yielded 4x better parameter efficiency than the baseline model.

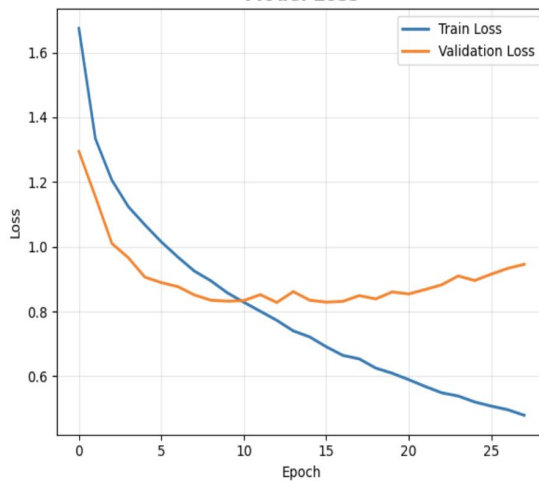
Evaluation (base_model)

MODEL EVALUATION

Model Accuracy

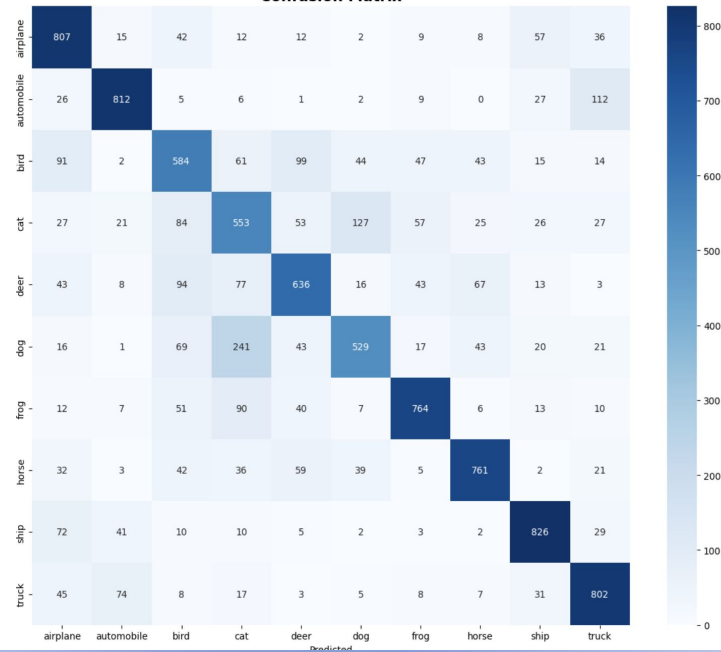


Model Loss



Final Test Results:
Test Accuracy: 0.7074
Test Loss: 0.8574

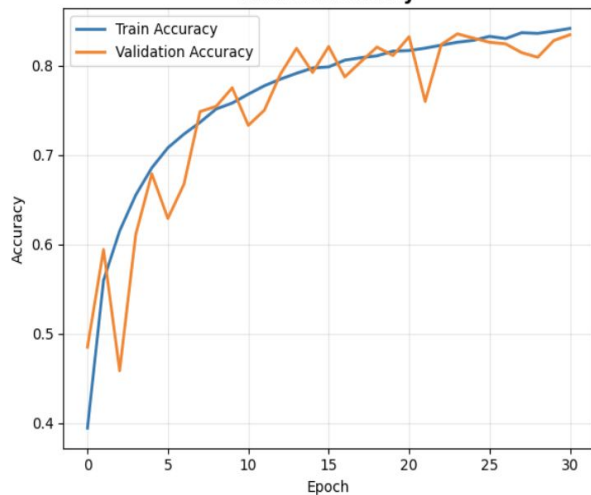
Confusion Matrix



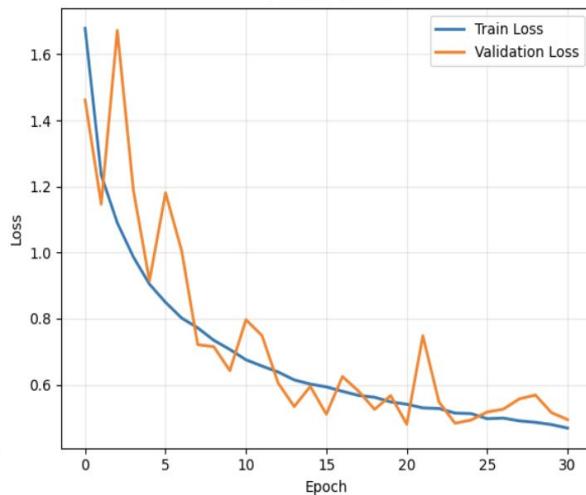
Evaluation (modern_vgg_compact.)

MODEL EVALUATION

Model Accuracy

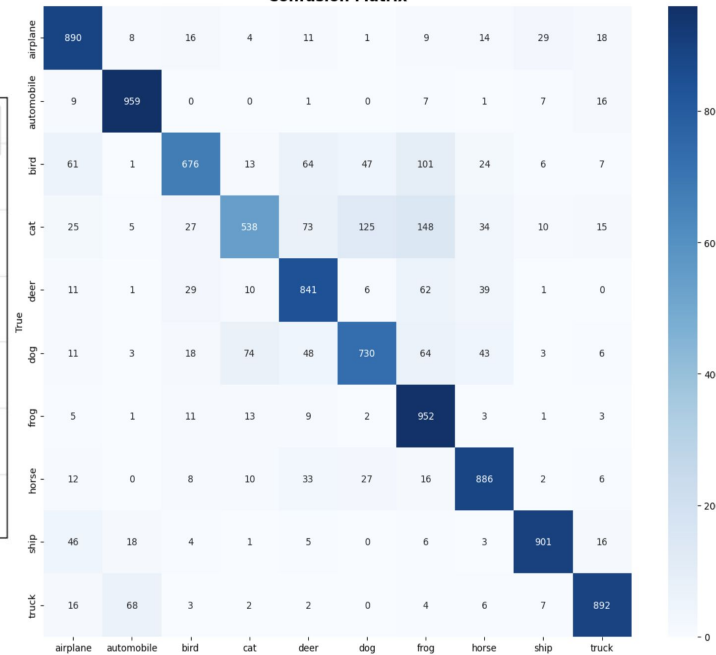


Model Loss



Final Test Results:
Test Accuracy: 0.8265
Test Loss: 0.5055

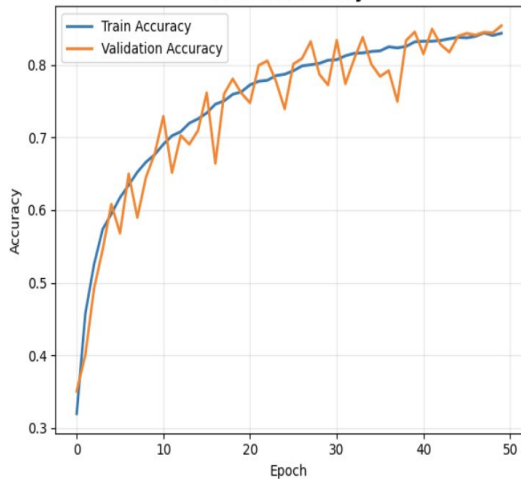
Confusion Matrix



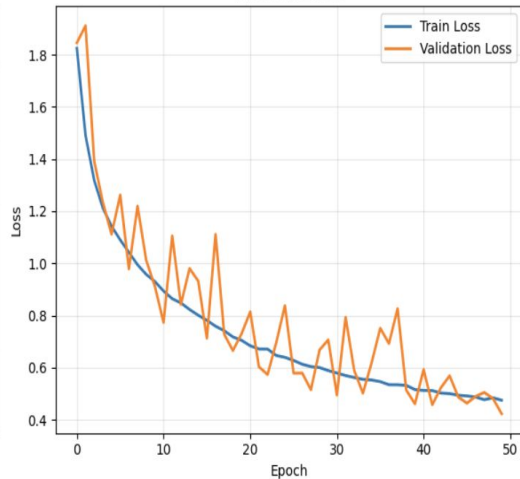
Evaluation (residual_blocks_cnn.ipynb)

MODEL EVALUATION

Model Accuracy

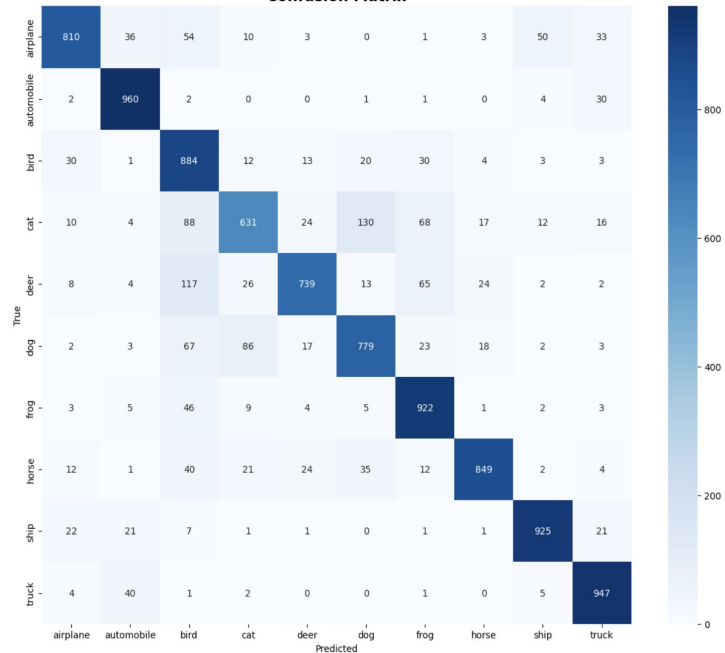


Model Loss

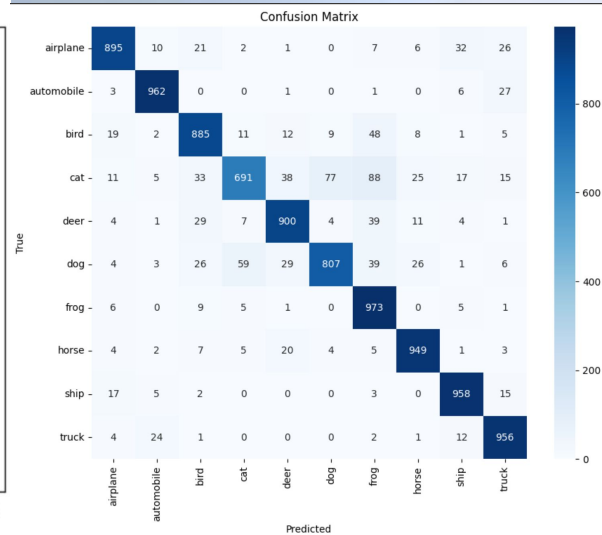
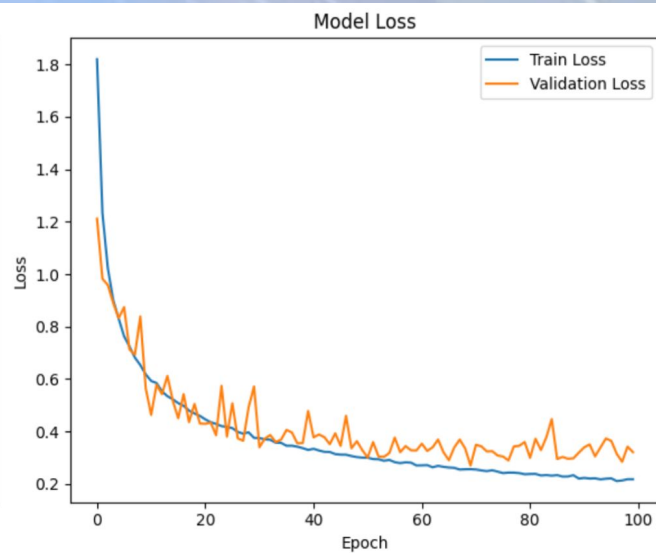
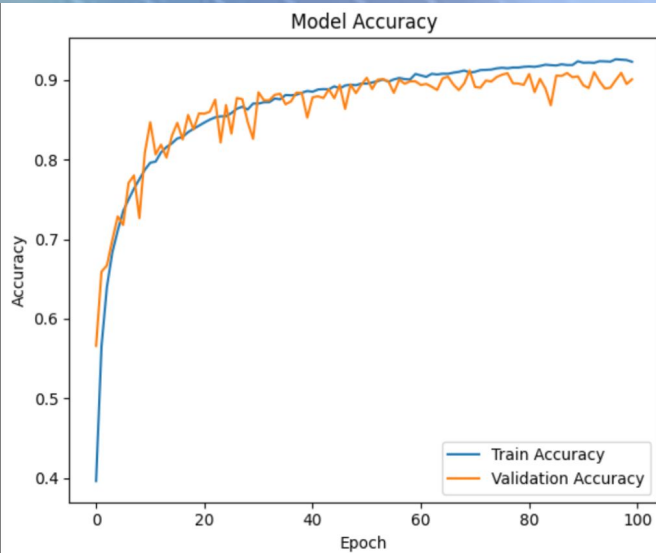


Final Test Results:
Test Accuracy: 0.8446
Test Loss: 0.4605

Confusion Matrix



Evaluation (vgg_style_with_batchnorm.ipynb)



Deployment

🎯 Objective

To make my trained AI model available for anyone to use through a simple and interactive web interface. The model can classify images into 10 categories based on the **CIFAR-10 dataset**:

⚙️ How It Works

- **Gradio** is a Python library that lets you turn AI models into usable web apps quickly.
- The code creates a frontend where users can:
 1. Upload an image.
 2. Get the model's top predictions instantly.

